

Sudoku genérico como CSP

Introdução

O Sudoku clássico é um exemplo canónico de **problema de satisfação de restrições (CSP)**: uma grelha $n^2 \times n^2$ onde cada linha, cada coluna e cada bloco $n \times n$ tem de conter os valores de 1 a n^2 sem repetições. A regra é sempre a mesma ("todos diferentes"); o que muda de grupo para grupo é apenas **que células** o compõem. Neste trabalho modelamos o problema com uma abstração única, a classe `box`, que representa um conjunto de células, algumas possivelmente fixas a um valor. Linhas, colunas e blocos são casos particulares (`path` e `cube`), e as pistas aleatórias são, outra vez, uma `box`. O modelo recebe um número arbitrário de grupos e trata-os todos da mesma forma, sem saber de onde vieram.

Escolha do solver

Começamos por resolver o problema com o **Z3**, através do pySMT, porque foi o solver que aprendemos nas aulas e aquele com que nos sentíamos mais à vontade. Mais tarde aprendemos a usar o **CP-SAT** do OR-Tools, e adaptamos o código que já tínhamos para o Z3. Como a classe `box` e os grupos não dependem do solver (só o modelo fala com a biblioteca), a adaptação limitou-se à classe do modelo, e pudemos comparar as duas abordagens.

Imports

```
In [ ]: import marimo as mo
        from pysmt.shortcuts import Symbol, Solver, And, LE, GE, Int, Equals,
        AllDifferent
        from pysmt.typing import INT
        import random
        from ortools.sat.python import cp_model
        import time
```

Classes

Classe box (R1)

A classe `box` é uma classe genérica que representa um conjunto de células de uma grelha $n^2 \times n^2$. Esta classe possui os métodos:

- `add(self, i, j, val=None)`, que acrescenta um certo valor de 1 a n^2 ou None à grelha, dentro das coordenadas (i,j), com i,j pertencentes a $[0, n^2-1]$. Caso as coordenadas ou o valor sejam inválidos, este é rejeitado.
- `to_matrix(self)`, que converte o dicionário `cells` numa matriz $n^2 * n^2$. Caso o valor com chave (i,j) não existe em `cells`, é colocado um 0 no lugar.

```
In [ ]: # R1
class box:

    def __init__(self, n, cells=None):
        self.n = n
        self.size = n * n
        self.cells = {}
        for (i, j), val in (cells or {}).items():
            self.add(i, j, val)

    def add(self, i, j, val=None):
        if not (0 <= i < self.size and 0 <= j < self.size):
            raise ValueError(f"Célula ({i}, {j}) fora da grelha {self.size}x{self.size}")
        if val is not None and not (1 <= val <= self.size):
            raise ValueError(f"Valor {val} fora de [1, {self.size}]")
        self.cells[(i, j)] = val

    def to_matrix(self):
        m = [[0] * self.size for _ in range(self.size)]
        for (i, j), val in self.cells.items():
            if val is not None:
                m[i][j] = val
            else:
                m[i][j] = 0
        return m
```

Classe cube (R2)

Um grupo que representa o bloco $i \times j$ cujo canto superior esquerdo é a célula $(i \times n, j \times n)$, parametrizado pelos índices de bloco (i,j) com $0 \leq i, j < n$

```
In [ ]: # R2
class cube(box):

    def __init__(self, n, bi, bj):
        if not (0 <= bi < n and 0 <= bj < n):
            raise ValueError(f"Índices de cube fora de [0, {n-1}]")
        super().__init__(n)
        for di in range(n):
            for dj in range(n):
                self.add(bi * n + di, bj * n + dj)
```

Classe path (R3)

Um grupo que representa o troço reto (horizontal ou vertical) de células entre duas coordenadas início e fim, inclusive. Não representa necessariamente uma linha ou coluna, podendo apenas representar um segmento mais pequeno.

```
In [ ]: # R3
class path(box):

    def __init__(self, n, inicio, fim):
        super().__init__(n)
        (i0, j0), (i1, j1) = inicio, fim
        if i0 != i1 and j0 != j1:
            raise ValueError("início e fim têm de estar na mesma linha ou
coluna")
        di = (i1 > i0) - (i1 < i0)
        dj = (j1 > j0) - (j1 < j0)
        i, j = i0, j0
        self.add(i, j)
        while (i, j) != (i1, j1):
            i, j = i + di, j + dj
            self.add(i, j)
```

Classe model (R5)

Uma classe que aceita dois solvers, z3 e CP-SAT (CP-SAT por defeito).

Esta classe possui os métodos:

- `add_groups(self, boxes)`, que recebe um número arbitrário de grupos (box, cube, path, ou pistas aleatórias) e, para cada um, impõe que as suas células sejam todas diferentes e fixa as que tiverem valor atribuído, neste problema apenas atribuiremos valores às pistas aleatórias.
- `solve(self)`, que devolve um sudoku resolvido caso haja solução, ou None caso contrário.
- `imprime_box_aleatorio(self, N)`, que imprime a pista aleatória, o ponto de partida para a resolução do sudoku.

```
In [ ]: # R5
class model:
    def __init__(self, n, solver=""):
        self.size = n * n
        self.box_inicial = None
        self.solver_name = solver

        if self.solver_name == "z3":
            self.solver = Solver(name="z3")
            self.x = {(i, j): Symbol(f"x_{i}_{j}", INT) for i in
range(self.size) for j in range(self.size)}
            for s in self.x.values():
                self.solver.add_assertion(And(GE(s, Int(1)), LE(s,
Int(self.size))))
        else:
```

```

self.model = cp_model.CpModel()
self.solver = cp_model.CpSolver()
self.x = {
    (i, j): self.model.NewIntVar(1, self.size, f"x_{i}_{j}")
    for i in range(self.size)
    for j in range(self.size)
}

def add_groups(self, boxes):
    for b in boxes:
        if self.solver_name == "z3":
            if isinstance(b, (path, cube)):
                self.solver.add_assertion(AllDifferent([self.x[c] for c
in b.cells]))
            else:
                self.box_inicial = b
                for c, val in b.cells.items():
                    if val is not None:
                        self.solver.add_assertion(Equals(self.x[c],
Int(val)))
        else:
            if isinstance(b, (path, cube)):
                self.model.AddAllDifferent([self.x[c] for c in
b.cells])
            else:
                self.box_inicial = b
                for c, val in b.cells.items():
                    if val is not None:
                        self.model.Add(self.x[c] == val)

def solve(self):
    if self.solver_name == "z3":
        if not self.solver.solve():
            return None
        return [[self.solver.get_value(self.x[(i, j)]).constant_value()
for j in range(self.size)] for i in range(self.size)]
    else:
        status = self.solver.Solve(self.model)
        if status in (cp_model.OPTIMAL, cp_model.FEASIBLE):
            return [[self.solver.Value(self.x[(i,j)]) for j in
range(self.size)] for i in range(self.size)]
        else:
            return None

def imprime_box_aleatorio(self, N):
    print("Pista aleatória")
    print_matriz(N, self.box_inicial.to_matrix())
    print()

```

Funções

Geração Aleatória de pistas (R4)

```
def box_aleatorio(n, k=None)
```

Uma função que devolve um grupo box, com k células preenchidas ou n, caso k não seja fornecido. Devolve um box completamente aleatório, podendo este ser inválido para a resolução do sudoku.

```
In [ ]: # R4
def box_aleatorio(n, k=None):
    b = box(n)
    rep = k if k is not None else n
    todas_coord = [(i,j) for i in range(b.size) for j in range(b.size)]
    for i, j in random.sample(todas_coord, rep):
        b.add(i, j, random.randint(1,b.size))
    return b
```

Resolução (R6)

```
def criar_sudoku(n)
```

Uma função que retorna um grupo de n^2 linhas, n^2 colunas, n^2 de cubes e uma pista aleatória.

```
In [ ]: # R6
def criar_sudoku(n):
    s = n * n
    linhas = [path(n, (i, 0), (i, s - 1)) for i in range(s)]
    colunas = [path(n, (0, j), (s - 1, j)) for j in range(s)]
    blocos = [cube(n, i, j) for i in range(n) for j in range(n)]
    return linhas + colunas + blocos + [box_aleatorio(n)]
```

Verificação

```
def verificar_sudoku(n, sudoku, box)
```

Função que verifica se todos os elementos das linhas, colunas e cubes são diferentes e que os elementos da pista aleatória inicial estão presentes na matriz, com os valores e coordenadas iguais.

```
In [ ]: def verificar_sudoku(n, sudoku, box):
    ok = True
    size = n*n
    # verificar linhas e colunas
    for c in range(size):
        linhas = []
        colunas = []
        for d in range(size):
            # linhas
            if (co := sudoku[c][d]) in linhas:
                print(f"Valor {co} repetido na linha {c}")
                ok = False
```

```

        linhas.append(co)

    # colunas
    if (li := sudoku[d][c]) in colunas:
        print(f"Valor {li} repetido na coluna {c}")
        ok = False
    colunas.append(li)

# cubes
for c in range(n):
    for d in range(n):
        cube = []
        for e in range(n):
            for f in range(n):
                if (val := sudoku[e + c*n][f + d*n]) in cube:
                    print(f"Valor {val} repetido no cube {c*n + d}")
                    ok = False
                cube.append(val)

# verificar valores do box inicial
for (i,j), val in box.cells.items():
    if val != (atual := sudoku[i][j]):
        print(f"Valor inicial {val} nas coordenadas ({i},{j}) alterado para {atual}")
        ok = False

# verificar se os valores do sudoku são válidos
for c in range(size):
    for d in range(size):
        if not 1 <= (val := sudoku[c][d]) <= size:
            print(f"Valor {val} inválido em ({c},{d}")
            ok = False

return ok

```

def print_matriz(n, m)

Imprime o sudoku $n^2 * n^2$

```

In [ ]: def print_matriz(n, m):
        w = len(str(n*n))
        for c in range(n*n):
            for d in range(n):
                print(' ', end='')
                for e in range(n):
                    print(f"{m[c][d * n + e]:{w}}", end=' ')
                if (d + 1 != n):
                    print('|', end='')
            if (c + 1) % n == 0 and c + 1 < n * n:
                print('\n' + '_' * ((n+1)*n + (n-1) + (n*n*w)))
            else:
                print()

```

Uso de LLM

Neste trabalho, usamos o Claude e o ChatGPT como apoio neste trabalho. Em concreto:

- Explicação de classes e herança em Python, de `*args`, de listas por compreensão e do `pySMT`.
- Discussão de solvers (CP-SAT, Z3 e SCIP) e escolha do Z3 com o `pySMT`, por ser o que a UC ensina e do CP-SAT por ser o que a UC recomenda.
- Código: o LLM propôs a estrutura inicial das classes `box`, `cube`, `path` e `model`, e de funções auxiliares. O código foi todo revisado, adaptado para o trabalho e aprovado pelo grupo. A função `print_matriz` foi feita inteiramente pelo grupo. As restantes funções foram adaptadas ao problema.
- Ajuda a perceber erros (por exemplo, o `*` ao passar a lista de grupos, e a chamada do método na classe em vez do objeto).
- O Chat-GPT apenas foi usado para explicar as funções do CP-SAT.
- Escrita deste relatório.

[link para o chat do claude](#)

[link para o chat do Chat-GPT](#)

Testes

CP-SAT (n=2)

```
In [ ]: def _():
    inicio = time.perf_counter()
    N = 2
    m = model(N)
    m.add_groups(criar_sudoku(N))
    m.imprime_box_aleatorio(N)
    sudoku = m.solve()
    if sudoku is not None:
        print("Solução")
        print_matriz(N, sudoku)
        if verificar_sudoku(N, sudoku, m.box_inicial):
            print("Sudoku sem valores repetidos")
            fim = time.perf_counter()
            print(f"Tempo de execução: {fim - inicio:.4f}s")
    else:
        print("Sem solução")
    return
_()
```

Pista aleatória

0	0		0	0
0	3		0	0

0	0		0	0
0	0		0	4

Solução

4	2		1	3
1	3		4	2

3	4		2	1
2	1		3	4

Sudoku sem valores repetidos

Tempo de execução: 0.0097s

CP-SAT (n=3)

```
In [ ]: box_alea = box_aleatorio(3)
def _():
    inicio = time.perf_counter()
    N = 3
    m = model(N)
    m.add_groups(criar_sudoku(N)[: -1] + [box_alea])
    m.imprime_box_aleatorio(N)
    sudoku = m.solve()
    if sudoku is not None:
        print("Solução")
        print_matriz(N, sudoku)
        if verificar_sudoku(N, sudoku, m.box_inicial):
            print("Sudoku sem valores repetidos")
            fim = time.perf_counter()
            print(f"Tempo de execução: {fim - inicio:.4f}s")
    else:
        print("Sem solução")
    return
_()
```


Pista aleatória

0	0	0		0	0	0		0	0	0
0	0	0		0	0	0		0	0	0
0	0	0		0	0	0		0	0	0

0	0	0		0	0	0		0	0	0
0	0	0		0	0	0		0	0	4
0	0	0		0	0	0		0	0	0

0	0	0		0	0	0		0	0	0
0	0	0		0	0	5		0	0	0
0	0	0		0	4	0		0	0	0

Solução

1	3	2		8	5	7		4	6	9
5	7	8		4	6	9		1	2	3
9	4	6		1	2	3		7	8	5

4	5	7		3	9	8		2	1	6
8	1	3		6	7	2		5	9	4
2	6	9		5	1	4		8	3	7

3	2	4		7	8	6		9	5	1
7	9	1		2	3	5		6	4	8
6	8	5		9	4	1		3	7	2

Sudoku sem valores repetidos

Tempo de execução: 0.0895s

Z3 (n=3)

```
In [ ]: def _():
    inicio = time.perf_counter()
    N = 3
    m = model(N, "z3")
    m.add_groups(criar_sudoku(N)[: -1] + [box_alea])
    m.imprime_box_aleatorio(N)
    sudoku = m.solve()
    if sudoku is not None:
        print("Solução")
        print_matriz(N, sudoku)
        if verificar_sudoku(N, sudoku, m.box_inicial):
            print("Sudoku sem valores repetidos")
            fim = time.perf_counter()
            print(f"Tempo de execução: {fim - inicio:.4f}s")
    else:
        print("Sem solução")
    return
_()
```

Pista aleatória

0	0	0		0	0	0		0	0	0
0	0	0		0	0	0		0	0	0
0	0	0		0	0	0		0	0	0

0	0	0		0	0	0		0	0	0
0	0	0		0	0	0		0	0	4
0	0	0		0	0	0		0	0	0

0	0	0		0	0	0		0	0	0
0	0	0		0	0	5		0	0	0
0	0	0		0	4	0		0	0	0

Solução

7	5	3		4	6	2		9	8	1
6	9	4		7	8	1		2	5	3
2	8	1		3	5	9		6	4	7

3	4	6		8	2	7		1	9	5
5	1	8		6	9	3		7	2	4
9	7	2		5	1	4		3	6	8

4	6	7		2	3	8		5	1	9
8	2	9		1	7	5		4	3	6
1	3	5		9	4	6		8	7	2

Sudoku sem valores repetidos

Tempo de execução: 25.1458s

CP-SAT (n=7)

```
In [ ]: def _():
    inicio = time.perf_counter()
    N = 7
    m = model(N)
    m.add_groups(criar_sudoku(N))
    m.imprime_box_aleatorio(N)
    sudoku = m.solve()
    if sudoku is not None:
        print("Solução")
        print_matriz(N, sudoku)
        if verificar_sudoku(N, sudoku, m.box_inicial):
            print("Sudoku sem valores repetidos")
            fim = time.perf_counter()
            print(f"Tempo de execução: {fim - inicio:.4f}s")
    else:
        print("Sem solução")
    return
_()
```

Adicionar valores inválidos

```
In [ ]: def _():  
        b = box(2)  
        b.add(0,0,10)  
        return  
        _()
```

```
Traceback (most recent call last):  
  File "", line 6, in <module>  
    _()  
  File "", line 3, in _cell_rqHn_  
    b.add(0,0,10)  
  File "", line 15, in add  
    raise ValueError(f"Valor {val} fora de [1, {self.size}]")  
ValueError: Valor 10 fora de [1, 4]
```

Adicionar valores fora do sudoku

```
In [ ]: def _():  
        b = box(2)  
        b.add(-1,-1,1)  
        return  
        _()
```

```
Traceback (most recent call last):  
  File "", line 5, in <module>  
    _()  
  File "", line 3, in _cell_Gtn0_  
    b.add(-1,-1,1)  
  File "", line 13, in add  
    raise ValueError(f"Célula ({i}, {j}) fora da grelha {self.size}x{self.size}")  
ValueError: Célula (-1, -1) fora da grelha 4x4
```

Conclusão

Modelamos o Sudoku $n^2 \times n^2$ como um CSP, com uma abstração única (`box`) a partir da qual linhas, colunas, blocos e pistas aleatórias são tratados da mesma forma pelo modelo. Como só a classe do modelo depende do solver, foi possível resolver o problema com o **Z3** e com o **CP-SAT**, alterando apenas essa classe. O fluxo completo (pistas aleatórias, montagem dos grupos, resolução e validação) funcionou para $n = 1$ até $n = 7$. Quando as pistas aleatórias não têm solução, simplesmente dizemos que não tem solução. Quanto ao desempenho, CP-SAT foi consideravelmente muito mais rápido que o Z3.